Assignment 1 - Project Phase 1

Data Acquisition, Text Processing & Cleaning | Introduction to NLP

Team: Imrah Maqsood and Ajwa | **Repository:** https://github.com/imrahhassan12-netizen/NLP_Assignment_1 (tag **assignment-1**)

Member	Roll number
Imrah Maqsood	0293
Ajwa	TODO

1. Topic and Dataset

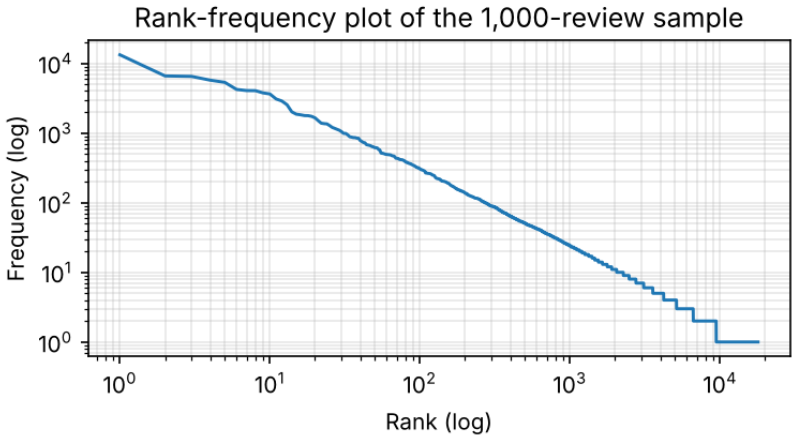
Topic: binary sentiment classification of movie reviews (Phase 2 will train classifiers on the cleaned text). **Why this dataset:** it is a public, well-known benchmark with a large amount of free-form English text, perfectly balanced labels, and real-world noise (HTML, contractions, ratings), so every cleaning decision has a visible effect.

Item	Value
Dataset	Large Movie Review Dataset (IMDB), Maas et al., 2011
Source	Zenodo record 7928582 (IMDB Dataset.csv); original release by Maas et al. (2011), Stanford
Size	50,000 labelled reviews (train + test splits combined; unlabelled reviews excluded)
Target	sentiment: positive / negative
Licence	Not specified in the Zenodo record; the original release is for research use with citation of Maas et al. (2011).
Environment	Python 3.12.3, pandas 3.0.6, NLTK 3.10.4, spaCy 3.8.18; virtual environment + requirements.txt
Raw data	Loaded into data/raw by src/download_dataset.py; never committed (data/ is git-ignored)

2. Exploration and Basic Statistics

Statistic	Value
Documents	50,000
Average / median length (words)	231.16 / 173
Shortest / longest review (words)	4 / 2,470
Target distribution	positive: 25,000 (50.0%), negative: 25,000 (50.0%)
Sample of 1,000 reviews: tokens / types	242,627 / 17,926 (type-token ratio 0.0739)
Hapax legomena (types seen once)	47.4% of types
Share of tokens covered by the 100 most frequent words	52.6%

Zipf / long tail. Most frequent words: the (13,263), a (6,593), and (6,498), of (5,732), to (5,337), is (4,217), br (4,066), it (4,054). The log-log rank-frequency curve is close to a straight line: a few function words dominate (the top 100 words cover 52.6% of all tokens) while 47.4% of the vocabulary occurs only once. This long tail of rare words is why normalisation (case folding, stemming) matters for a bag-of-words model. The raw top list also shows markup noise such as 'br'.



Examples of messy text found in the data

Issue	Reviews affected	Real example
HTML line-break tags	29,202 (58.4%)	ey are right, as this is exactly what happened with me. The first thing that struck me about Oz was its b
URLs	195 (0.39%)	'S GUIDE TO AN INCONVENIENT TRUTH" can be found at is :www.cei.org. A shorter 10-page version can be found at: www.cei.org
ALL-CAPS emphasis	11,339 (22.68%)	which totally ruins all the film! I expected to see a BOOGEYMAN similar movie, and instead i watched a drama with some
Repeated punctuation	12,662 (25.32%)	gangstas, Latinos, Christians, Italians, Irish and more....so scuffles, death stares, dodgy dealings and shady agr
Non-ASCII characters	4,660 (9.32%)	that has been done and redone so many times that clichéd dialogue, rehashed plot and over-the-top action seque
Irregular spacing	3,986 (7.97%)	rise really that in the 1990s, a glut of 'Die Hard on a .....' movies cashed in on the wrong guy, wrong place, w
Rating expressions	2,980 (5.96%)	ly hilarious at the same time, I'm giving this movie a 3/10, an improvement from the 0.5/10 I was originally going

3. Tokenization and Regular Expressions

Four tokenizers were compared: whitespace split, a regex tokenizer, NLTK's Treebank-style tokenizer and spaCy's English tokenizer rules.

Input	I couldn't believe it; the acting was top-notch, but the plot isn't great. Rated 7.5/10!	Check www.imdb.com for the cast's full list... they're amazing!!!	It's a feel-good movie (1994) that you'll love <U+1F60D>
Whitespace split	I | couldn't | believe | it; | the | acting | was | top-notch, | but | the | plot | isn't | great.<br | /><br | />Rated | 7.5/10!	Check | www.imdb.com | for | the | cast's | full | list... | they're | amazing!!!	It's | a | feel-good | movie | (1994) | that | you'll | love | <U+1F60D>
Regex \b\w+\b	I | couldn | t | believe | it | the | acting | was | top | notch | but | the | plot | isn | t | great | br | br | Rated | 7 | 5 | 10	Check | www | imdb | com | for | the | cast | s | full | list | they | re | amazing	It | s | a | feel | good | movie | 1994 | that | you | ll | love
NLTK (Treebank-style)	I | could | n't | believe | it | ; | the | acting | was | top-notch | , | but | the | plot | is | n't | great. | < | br | / | > | < | br | / | > | Rated | 7.5/10 | !	Check | www.imdb.com | for | the | cast | 's | full | list | ... | they | 're | amazing | ! | ! | !	It | 's | a | feel-good | movie | (| 1994 |) | that | you | 'll | love | <U+1F60D>
spaCy (English rules)	I | could | n't | believe | it | ; | the | acting | was | top | - | notch | , | but | the | plot | is | n't | great.<br | /><br | />Rated | 7.5/10 | !	Check | www.imdb.com | for | the | cast | 's | full | list | ... | they | 're | amazing | ! | ! | !	It | 's | a | feel | - | good | movie | (| 1994 |) | that | you | 'll | love | <U+1F60D>

Tokenizer	Tokens (1000-review sample)	Types (lowercased)
Whitespace split	233,773	32,528
Regex \b\w+\b	242,627	17,926
NLTK (Treebank-style)	274,973	22,438
spaCy (English rules)	272,809	20,009

Whitespace splitting leaves punctuation glued to words ('great.<br', 'amazing!!!'). The regex tokenizer removes punctuation but breaks contractions and hyphenated words apart (couldn't becomes couldn + t). NLTK and spaCy keep punctuation as separate tokens and split contractions into a stem plus a clitic (could + n't), which keeps the negation recoverable. None of them understands HTML (the NLTK output shows '<', 'br', '/', '>'), so markup must be removed before tokenizing.

Regular expressions written for this data

Name	Pattern	Explanation	Reviews matching	Example
HTML tags	<[^>]+>	Matches '<', any run of characters that is not '>', then '>'. Finds line breaks so they can be deleted instead of becoming a 'br' token.	29,202	
URLs	https?://\S+|www\.\S+	Matches http(s):// or www. followed by non-space characters. Web links carry no sentiment.	195	www.cei.org.
Rating expressions	\b(\d{1,2})(?:\.\d)?\s*/\s*10\b	Matches 1-2 digits, an optional decimal, optional spaces and '/10'. Keeps scores such as 7/10 as a single useful token (rating7).	2,980	3/10
Repeated punctuation	([!?.])\1{2,}	Matches a !, ? or . repeated 3+ times via a back-reference. Marks emphatic style.	12,662	....
Negation contractions	\b\w+n't\b	Matches word characters followed by n't. Locates words such as didn't and isn't whose negation must survive cleaning.	29,586	wouldn't

Tokenization challenges in this data. (1) **Contractions and negation:** 29,586 reviews contain n't forms; splitting them carelessly leaves stray tokens such as 't' or 'll' and can lose 'not', which flips sentiment. (2) **Markup and hyphenation:** 29,202 reviews contain HTML tags that would create a 'br' token, and 26,821 contain hyphenated words (feel-good) whose parts mean something different alone. (3) **Rare symbols:** 4,660 reviews contain non-ASCII characters or emoji, which whitespace and regex tokenizers treat inconsistently.

4. Cleaning and Normalisation Pipeline

All steps live in one function, `clean_text()` in src/clean_text.py, applied identically to every review in this order:

#	Step	Justification for this data
1	Unicode NFKC normalisation	Done first so visually identical characters (curly quotes, full-width forms) become one code point before any regex runs. English text, so no script-specific rules are needed.
2	HTML entity decoding and tag removal	Reviews contain tags; decoding entities first ensures escaped tags are also caught. Tags are markup, not opinion.
3	URL and e-mail removal	Links carry no sentiment and inflate the vocabulary with unique strings.
4	Rating normalisation (7/10 to rating7)	A score is strong sentiment evidence; this keeps it as one token instead of the fragments 7 and 10.
5	Case folding (lowercase)	Merges The/the; shrinks the vocabulary. Trade-off: ALL-CAPS emphasis is lost, accepted because the effect is small relative to vocabulary sparsity.
6	Contraction expansion	Run after lowercasing and before tokenizing so n't becomes 'not' and is kept, instead of leaving junk tokens.
7	Regex tokenization [a-z]+\d*|\d+	Drops punctuation and symbols; keeps words and numbers.
8	Stop-word removal (NLTK list, 187 words)	Removes function words. Words that carry sentiment or negation are protected: against, but, few, more, most, never, no, nor, not, only, too, very.
9	Porter stemming	See below.

Stemming vs lemmatization. Porter stemming was chosen. Reviews are long (50k documents) and Phase 2 uses bag-of-words style features, so merging inflected forms matters more than producing dictionary words. Stemming is fast, deterministic and needs no POS tagger or model download; lemmatization without POS tags leaves verbs unchanged (watching stays watching) and with a tagger is much slower. The cost is non-words such as 'movi'. Examples: watching -> watch, watched -> watch, movies -> movi, acting -> act, loved -> love, loving -> love, disappointing -> disappoint, disappointed -> disappoint, terrible -> terribl, wonderfully -> wonder.

Vocabulary	Size
Before cleaning (raw text, case-sensitive \w+ tokens)	130,065
Before cleaning (raw text, lowercased)	101,944
After cleaning	70,010
Reduction	46.2% vs case-sensitive raw, 31.3% vs lowercased raw
Rows kept / dropped (empty after cleaning)	50,000 / 0

Before	After
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked. They are right, as this is exactly what happened with me. The first thing that struck me about Oz was its brutality and unflinching scenes of violence, which set in right from the word GO. Trust me, this is not a show for	one review mention watch 1 oz episod hook right exactli happen first thing struck oz brutal unflinch scene violenc set right word go trust not show faint heart timid show pull no punch regard drug sex violenc hardcor classic use word call oz nicknam given oswald maximum secur state penitentari focus mainli emerald citi experimen

5. Reproducibility

```
git clone <repo-url> && cd NLP_Assignment_1 && git checkout assignment-1
python -m venv .venv && source .venv/bin/activate   (Windows: .venv\Scripts\activate)
pip install -r requirements.txt
python run_all.py        (download, explore, tokenize, clean, build report)
Output: data/processed/cleaned_reviews.csv  (columns: cleaned_review, sentiment)
```

Exact tagged commit: https://github.com/imrahhassan12-netizen/NLP_Assignment_1 at tag **assignment-1**. The seed is fixed (42) so sampling and results are identical on every run.

6. Execution Evidence (console output of each script)

Exploration

```
Documents            : 50,000
Average length       : 231.16 words (min 4, median 173, max 2,470)
Target distribution  : {'positive': 25000, 'negative': 25000}
Sample (1000 reviews): tokens=242,627, types=17,926, TTR=0.0739
Hapax legomena       : 47.4% of types
Top-100 words cover  : 52.6% of tokens
Most frequent words  : the(13263), a(6593), and(6498), of(5732), to(5337), is(4217), br(4066), it(4054)
Messy-text findings  :
  HTML line-break tags   29,202 reviews (58.4%)
```

Tokenization

```
Tokenizer comparison on sample sentence 1:
  Whitespace split         ['I', "couldn't", 'believe', 'it;', 'the', 'acting', 'was', 'top-notch,', 'but', 'the', 'plot', "isn'
  Regex \b\w+\b            ['I', 'couldn', 't', 'believe', 'it', 'the', 'acting', 'was', 'top', 'notch', 'but', 'the', 'plot', '
  NLTK (Treebank-style)    ['I', 'could', "n't", 'believe', 'it', ';', 'the', 'acting', 'was', 'top-notch', ',', 'but', 'the', '
  spaCy (English rules)    ['I', 'could', "n't", 'believe', 'it', ';', 'the', 'acting', 'was', 'top', '-', 'notch', ',', 'but',
Corpus counts on 1000-review sample:
  Whitespace split         tokens=233,773 types=32,528
  Regex \b\w+\b            tokens=242,627 types=17,926
  NLTK (Treebank-style)    tokens=274,973 types=22,438
```

Cleaning

```
Cleaning completed
Rows in / out        : 50,000 / 50,000
Vocabulary (raw)     : 130,065
Vocabulary (lowercase raw): 101,944
Vocabulary (cleaned) : 70,010
Reduction vs raw     : 46.2%
Stem examples        : watching-&gt;watch, watched-&gt;watch, movies-&gt;movi, acting-&gt;act, loved-&gt;love, loving-&gt;love,
Saved                : /home/claude/final/NLP_Assignment_1/data/processed/cleaned_reviews.csv
```

7. Contribution Statement

Member	Roll number	Contribution
Imrah Maqsood	0293	Dataset exploration, tokenization comparison, text preprocessing, implementation, testing and report preparation.
Ajwa	TODO	Project development, dataset exploration, text preprocessing, testing and documentation.